

Introduction to Reinforcement Learning

Majid Ghasemi^{*,1} and Dariush Ebrahimi¹

¹Department of Computer Science, Wilfrid Laurier University, Waterloo, Canada

{mghasemi^{*}, debrahimi}@wlu.ca

^{*}Corresponding author

Abstract

Reinforcement Learning (RL), a subfield of Artificial Intelligence (AI), focuses on training agents to make decisions by interacting with their environment to maximize cumulative rewards. This paper provides an overview of RL, covering its core concepts, methodologies, and resources for further learning. It offers a thorough explanation of fundamental components such as states, actions, policies, and reward signals, ensuring readers develop a solid foundational understanding. Additionally, the paper presents a variety of RL algorithms, categorized based on the key factors such as model-free, model-based, value-based, policy-based, and other key factors. Resources for learning and implementing RL, such as books, courses, and online communities are also provided. By offering a clear, structured introduction, this paper aims to simplify the complexities of RL for beginners, providing a straightforward pathway to understanding and applying real-time techniques.

1 Introduction

Reinforcement Learning (RL) is a subfield of Artificial Intelligence (AI) that focuses on training by interacting with the environment, aiming to maximize cumulative reward over time [1]. In contrast to supervised learning, where the objective is to learn from labeled examples, or unsupervised learning, which is based on detecting patterns in the data, RL deals with an autonomous agent that must make intuitive decisions and consequently learn from its actions, often without existing data. The key idea is to learn how the world works (e.g., what action gets a (positive) reward and which does not) to maximize cumulative rewards over time through trial-and-error exploration. RL revolves around several key concepts: **States**, **Actions**, **Policy**, **Rewards**, **Transition Dynamics** (Probabilities), and **Environment Model**. Each of these components plays a crucial role in defining the agent's interaction with its environment and the overall learning process, and are defined over the next paragraphs. This paper assumes readers to have basic knowledge of Machine Learning (ML) algorithms like Supervised and Unsupervised Learning.

A **State** ($s \in \mathcal{S}$) represents a specific condition or configuration of the environment at a given time as perceived by the agent. A state sets the scene for the agent to make choices and select actions, which describes all states from which an agent can choose for each action that occurs. For example, in chess, a state might be one specific layout of pieces on the board. In brief, states are the situations that an agent can be in and observe to make decisions. States can be discrete (the given example) and continuous (e.g., the position of a robot, in terms of x and y coordinates). **Actions** ($a \in \mathcal{A}$) are the set of possible moves or decisions an agent can make while interacting with the environment. A selected action is part of the strategy followed by an agent to reach its desired goals according to its current states and policy. In the chess example, moving a piece from one square to another is an action. Similar to states, actions can be discrete and continuous. The given chess example has a set of discrete actions (moving the piece according to the rules, which has a finite set of actions). For continuous actions, let us consider the robot example. A robot can change its coordinates and its movement speed, which both are continuous. **Policy** (π) guides the behavior of a learning agent by mapping perceived states of the environment into actions. This could be a simple function, a lookup table, or a complex computation, e.g., approximated by Neural Networks. Policies can be stochastic, defining the likelihood of taking certain actions. A *stochastic policy* ($\pi(a|s)$) defines a probability distribution over possible actions for a given state. Instead of selecting a single action, it

samples actions based on the probabilities defined by the policy. On the other hand, A *deterministic policy* ($\pi(s) = a$) directly maps a state to a specific action. For a given state, the action taken is fixed and does not involve any randomness. In the game of chess, an agent’s policy determines the move it makes given the current board configuration (state). Imagine an agent evaluating its options in a given state of the board. It assigns probabilities to each legal move based on their estimated utility. For instance: $\pi(a_1|s) = 0.5$, $\pi(a_2|s) = 0.3$, $\pi(a_3|s) = 0.2$. Here, a_1 , a_2 , and a_3 are possible moves, and the policy specifies a 50% chance of selecting a_1 , 30% for a_2 , and 20% for a_3 . The agent chooses its move by sampling from this distribution. This approach introduces diversity in decision-making, which can be particularly useful when exploring novel strategies or dealing with uncertain opponents. Conversely, a deterministic policy maps the current state s to a specific move a (e.g., $\pi(s) = a_1$). In this case, the agent always chooses a_1 (e.g., moving the queen to a specific square) when the board configuration matches s . Deterministic policies are efficient in well-defined and predictable scenarios, such as when the agent has already learned the optimal moves for most board configurations. **Rewards** ($r \in \mathcal{R}$) are a critical factor in RL since it provides the agent with an objective at each time step, defining both local and global goals that the agent aims to achieve over time. Rewards differentiate positive from negative events and help update policies according to the outcomes of actions. Rewards depend on the state of the environment and the actions taken. For instance, in a Chess scenario, winning the game will lead to a positive reward, while losing points will lead to a negative reward. Also, the game might end up in a draw, which lead to no reward. Reward function defines the immediate reward obtained when taking action a in state s . It can be deterministic ($\mathcal{R}(s, a) = r$) or stochastic ($P(r|s, a)$). **Transition Dynamics** function defines the probability of reaching a new state s' given the current state s and action a . It can be written mathematically as $\mathcal{P}(s'|s, a)$. **Environment Model** is an approximation or prediction of the environment, mapping out what will be available (the next state and reward) given a particular input from the state and action. These models help with planning by identifying what actions should be taken based on possible future events. RL approaches that use models are called *model-based* methods, whereas those relying solely on trial-and-error learning are *model-free* methods. This characteristic is an important factor to choose the appropriate algorithms, suitable for the problem. The environment’s dynamics can be represented as a transition function and a reward function. Together, these form the Markov Decision Process (MDP):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$$

where $\mathcal{S}$ is state space, $\mathcal{A}$ is action space, $\mathcal{P}(s'|s, a)$ is the transition dynamics, $\mathcal{R}(s, a)$ is the Reward function, and $\gamma \in [0, 1]$. γ is the discount factor, a parameter that determines how much importance is given to future rewards compared to immediate rewards.

Figure 1 illustrates the interaction between an RL agent and its environment. The agent observes the current state ($\mathcal{S}_t$), selects an action ($\mathcal{A}_t$) based on its policy (π), and receives a reward based on the reward distribution ($\mathcal{R}_t$) along with the next state ($\mathcal{S}_{t+1}$). This feedback loop is critical for the agent to learn and update its policy to maximize cumulative rewards.

The rest of the paper is organized as follows: Section 2 introduces backgrounds and key concepts in RL, starting with multi-armed bandit. Bandits are a great place to start learning RL’s foundational concepts like value functions and the Bellman equations. A complete introduction to core RL methods is given in section 3. Section 4 analyzes essential RL algorithms, categorized them in a comprehensive manner. In section 5, useful resources for learning RL are provided for readers would like to delve deeper in the realm of RL. Finally, section 6 Concludes the paper.

2 Backgrounds & Key Concepts

Understanding RL requires a solid grasp of its foundational principles and the mathematical frameworks that guide agent-environment interactions. This section introduces the essential building blocks of it, starting with the simplest form of decision-making under uncertainty: the Multi-Armed Bandit problem. This provides an intuitive gateway to understanding how agents learn from evaluative feedback. Building on this, we delve into MDPs, which formalize sequential decision-making by balancing immediate and future rewards. Finally, we discuss key RL metrics, such as value functions and policies, which form the backbone

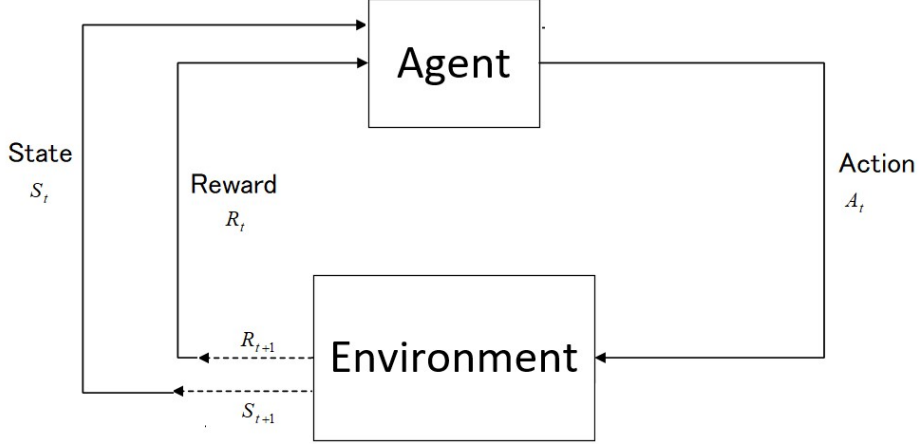


Figure 1: Overview of Reinforcement Learning [1]

of most RL algorithms. Together, these topics lay the groundwork for exploring the core RL methods in the next section.

2.1 Multi-Armed Bandit(s)

The Multi-Armed Bandit problem serves as a foundational example for understanding Reinforcement Learning (RL). It represents a simplified decision-making scenario where an agent repeatedly chooses from $\mathcal{K}$ actions (or arms) to maximize cumulative rewards over time. Each action is associated with an unknown reward distribution, requiring the agent to balance *exploration* (gathering information about all actions) and *exploitation* (maximizing immediate rewards using the best-known action). Unlike supervised learning, where instructive feedback explicitly points to correct actions, RL often involves evaluative feedback. This type of feedback only signals the quality of chosen actions, making it harder to identify optimal strategies without extensive interaction with the environment.

In the $\mathcal{K}$ -armed bandit problem, each action $a \in \{a_1, a_2, \dots, a_k\}$ yields a reward drawn from a stationary probability distribution. The *stationarity* assumption implies that the reward probabilities remain constant over time [2]. For each of the $k \in \mathcal{K}$ actions available, there is an expected average reward, referred to as the value of the action [1]. Let $\mathcal{A}_t$ denote the action chosen at time step t , and $\mathcal{R}_t$ the corresponding reward received. The true value of an action, $q_*(a)$, is defined as the expected reward by taking that action as:

$$q_*(a) = \mathbb{E}[\mathcal{R}_t \mid \mathcal{A}_t = a] \quad (1)$$

If $q_*(a)$ were known for all a , the problem would be trivial: the agent would always choose the action with the highest $q_*(a)$. However, in practice, these values are unknown and must be estimated through interaction with the environment. After estimating action values, there is at least one action with the highest estimated value at each time step. These are called greedy actions. Selecting a greedy action exploits current knowledge for immediate reward, while selecting non-greedy actions explores to improve estimates. Balancing exploration and exploitation is crucial in RL. Opting for exploitation maximizes immediate reward, while exploration can yield higher overall reward depending on various factors [1, 3]. Estimating action values through sampling averages is efficient for stationary bandit problems. In real-world problems with non-stationary environments, it makes sense to weight recent rewards more. Using a constant step-size parameter is popular. The following update rule can be used to update the action value, which is known as **sample-average method**.

$$\mathcal{Q}_t(a) = \frac{\sum_{i=1}^{t-1} \mathcal{R}_i \cdot \mathbf{1}_{\{\mathcal{A}_i=a\}}}{\sum_{i=1}^{t-1} \mathbf{1}_{\{\mathcal{A}_i=a\}}}, \quad (2)$$

where $\mathbf{1}_{\{\mathcal{A}_i=a\}}$ is an indicator function that equals 1 if action a was chosen at step i , and 0 otherwise. Over time, $\mathcal{Q}_t(a)$ converges to $q_*(a)$ by the law of large numbers [4], provided all actions are sampled sufficiently often.

$$\mathbf{1}_{\{\mathcal{A}_i=a\}} = \begin{cases} 1 & \text{if action } a \text{ is taken at time step } i \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

Equation 2 can be written in a different way as follows, which represents the action value $\mathcal{Q}_t(a)$ as the average of the rewards received from that action up to time step t .

$$\mathcal{Q}_t(a) = \frac{\text{sum of rewards when the action } a \text{ is taken prior to time step } t}{\text{number of times the action } a \text{ is taken prior to time step } t} \quad (4)$$

$$\mathcal{Q}_{n+1} = \mathcal{Q}_n + \alpha[\mathcal{R}_n - \mathcal{Q}_n], \quad (5)$$

To iteratively refine $\mathcal{Q}_t(a)$, we use Equation 5, known as **incremental update rule**, where $\mathcal{Q}_{n+1}$ is the updated action value after observing reward $\mathcal{R}_n$. $\mathcal{Q}_n$ is the previous estimate, and α is the learning rate (a step-size parameter determining how much new information overrides the old estimate). This update rule incrementally adjusts the action value based on the difference between the received reward and the current estimate, weighted by the learning rate α . This iterative process allows the agent to refine its value estimates and improve decision-making over time. In other words, $\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{StepSize}[\text{Target} - \text{OldEstimate}]$ [1].

One of the critical challenges in solving the multi-armed bandit problem, and in RL as well, is balancing exploration and exploitation, the greedy action selection ($\mathcal{A}_t = \arg \max_a \mathcal{Q}_t(a)$). Several strategies address this trade-off. We touch some of the widely used ones here, starting with ϵ -**greedy**. In this method, with probability ϵ , choose a random action (exploration); otherwise, select the greedy action (exploitation). This ensures that all actions are sampled, albeit less frequently for suboptimal actions. Another method, known as **Upper Confidence Bound (UCB)**, which balances exploitation and exploration by incorporating uncertainty into action selection [5]:

$$\mathcal{A}_t = \arg \max_a \left[\mathcal{Q}_t(a) + c \sqrt{\frac{\ln t}{\mathcal{N}_t(a)}} \right], \quad (6)$$

where $\mathcal{N}_t(a)$ is the number of times action a has been chosen, and $c > 0$ controls the exploration rate. UCB prioritizes actions with high uncertainty or fewer samples. **Optimistic Initialization**, assigns high initial values to $\mathcal{Q}_1(a)$, encouraging exploration of all actions early on. For example, setting $\mathcal{Q}_1(a) = +5$ makes untried actions appear attractive initially.

These methods ensure all actions are sampled enough to estimate their true value accurately, making the selection of the best action almost certain over time. However, these are theoretical long-term benefits and may not directly indicate practical effectiveness [6]. To begin with, estimating action values by sampling averages is inefficient for large numbers of samples ($\mathcal{K} > n$). A more efficient approach derives $\mathcal{Q}_n$ as:

$$\mathcal{Q}_n \equiv \frac{\mathcal{R}_1 + \mathcal{R}_2 + \dots + \mathcal{R}_{n-1}}{n-1} \quad (7)$$

We can express the update rule for action value recursively

$$\mathcal{Q}_{n+1} = \mathcal{Q}_n + \frac{1}{n}[\mathcal{R}_n - \mathcal{Q}_n] \quad (8)$$

This recursive equation requires memory only for $\mathcal{Q}_n$ and n , with minimal computation for each reward [7]. Sample averages apply to stationary bandit problems. In non-stationary environments, recent rewards are more relevant. Using a decaying constant step-size parameter, issues related to non-stationary problems can be mitigated [1].

The initial estimates of action values, $\mathcal{Q}_1(a)$, play a crucial role in the learning process. These initial guesses influence the early decisions made by the agent. While sample-average methods can reduce this initial bias after each action is chosen at least once, methods using a constant step-size parameter, α , tend

Algorithm 1 A simple bandit algorithm

```
1: Initialize:  
2: for  $a = 1$  to  $k$  do  
3:    $Q(a) \leftarrow 0$   
4:    $N(a) \leftarrow 0$   
5: end for  
6: Repeat forever:  
7:  $\mathcal{A} \leftarrow \begin{cases} \arg \max_a Q(a) & \text{with probability } 1 - \epsilon \\ \text{a random action} & \text{with probability } \epsilon \end{cases}$   
8:  $R \leftarrow \text{bandit}(\mathcal{A})$   
9:  $N(\mathcal{A}) \leftarrow N(\mathcal{A}) + 1$   
10:  $Q(\mathcal{A}) \leftarrow Q(\mathcal{A}) + \frac{1}{N(\mathcal{A})} [R - Q(\mathcal{A})]$ 
```

to mitigate this bias more gradually over time. Setting optimistic initial values can be particularly advantageous. By assigning higher initial estimates (e.g., +5), the agent is encouraged to explore more actions early on. This is because the initial optimism makes untried actions appear more attractive, thus promoting exploration even when the agent uses a greedy strategy. This approach helps ensure that the agent thoroughly investigates the action space before converging to a final policy. However, this strategy requires careful consideration in defining the initial values, which are often set to zero in standard practice. The choice of initial values should reflect an informed guess about the potential rewards, and overly optimistic values can prevent the agent from converging efficiently if not properly managed. Overall, optimistic initial values can be a useful technique to balance exploration and exploitation in RL, encouraging broader exploration and potentially leading to more optimal long-term policies [8].

Over the next subsection, another fundamental concept in RL, MDPs, are introduced.

2.2 Markov Decision Process (MDPs)

An MDP provides a framework for sequential decision-making in which actions affect immediate rewards as well as future outcomes. In MDPs, immediate rewards are balanced with delayed rewards. In contrast to bandit problems in which the goal is to determine the value of each action a , MDPs aim to measure the value of taking action a in state s , or the value of being in state s assuming optimal actions are taken. A correct assessment of the long-term effects of interventions requires the estimation of these state-specific values [1]. MDPs consist of states, actions, and rewards $(\mathcal{S}, \mathcal{A}, \mathcal{R})$. Discrete probability distributions are assigned to the random variables $\mathcal{R}_t$ and $\mathcal{S}_t$ based on the preceding state and action. Using the probabilities of occurrence of the random variables $\mathcal{R}_t$ and $\mathcal{S}_t$, derive equations for these variables. A system is considered Markovian when the outcome of an action is independent of past actions and states, relying solely on the current state [9]. The Markov property requires the state to encapsulate significant details of the entire past interaction influencing future outcomes [10]. This definition is the basis of MDPs being used in RL. To describe the dynamics of an MDP, we use the state-transition probability function $p(s', r \mid s, a)$, which is defined as follows:

$$p(s', r \mid s, a) \equiv \Pr\{\mathcal{S}_t = s', \mathcal{R}_t = r \mid \mathcal{S}_{t-1} = s, \mathcal{A}_{t-1} = a\} \quad (9)$$

where the function p defines the MDP dynamics. The following state-transition probabilities, state-action and state-action-next-state triple rewards can be derived from the four-argument dynamic function p . We can derive the state-transition probabilities, the expected reward for state-action pairs, and the expected rewards for state-action-next-state triples as follows:

$$p(s' \mid s, a) \equiv \Pr\{\mathcal{S}_t = s' \mid \mathcal{S}_{t-1} = s, \mathcal{A}_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r \mid s, a) \quad (10)$$

$$r(s, a) \equiv \mathbb{E}\{\mathcal{R}_t \mid \mathcal{S}_{t-1} = s, \mathcal{A}_{t-1} = a\} = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r \mid s, a) \quad (11)$$

$$r(s, a, s') \equiv \mathbb{E}\{\mathcal{R}_t \mid \mathcal{S}_{t-1} = s, \mathcal{A}_{t-1} = a, \mathcal{S}_t = s'\} = \sum_{r \in \mathcal{R}} \frac{rp(s', r \mid s, a)}{p(s' \mid s, a)} \quad (12)$$

The concept of actions encompasses any decisions relating to learning, and the concept of states encompasses any information that is available in order to inform those decisions. As part of the MDP framework, goal-directed behavior is abstracted through interaction. Any learning problem can be reduced to three signals between an agent and its environment: actions, states, and rewards. A wide range of applications have been demonstrated for this framework [1]. We are now able to formally define and solve RL problems. We have defined rewards, objectives, probability distributions, the environment, and the agent. Some concepts, however, were defined informally. According to our statement, the agent seeks to maximize future rewards, but how can this be mathematically expressed? The return, denoted G_t , is the cumulative sum of rewards received from time step t onwards. For episodic tasks, it is defined as follows:

$$\mathcal{G}_t \equiv \mathcal{R}_{t+1} + \mathcal{R}_{t+2} + \dots + \mathcal{R}_{\mathcal{T}} \quad (13)$$

Here, $\mathcal{G}_t$ is a specific function of the reward sequence. Episodic problems are those in which the interactions between agents and their environment occur naturally in sequence, known as episodes, and tasks are termed episodic tasks. The game hangman is a good example of this. At the end of each episode, a standard starting state is restored. The term *new games* refers to the next state after the terminal state, which is the final state leading to the end of an episode. It is common for ongoing tasks to involve interactions that persist continuously throughout the duration of the task, such as process control or applications that utilize robots with prolonged lifespans. The term *continuing tasks* refers to these activities. As there are no terminal states in continuing tasks ($\mathcal{T} = \infty$), the return for continuing tasks should be defined differently. It is possible that the return could be infinite if the agent consistently receives a reward. For continuing tasks, where there is no terminal state, the return $\mathcal{G}_t$ is defined as the discounted sum of future rewards:

$$\mathcal{G}_t \equiv \mathcal{R}_{t+1} + \gamma \mathcal{R}_{t+2} + \gamma^2 \mathcal{R}_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k \mathcal{R}_{t+k+1} \quad (14)$$

where γ is the discount rate ($0 \leq \gamma \leq 1$). The discount rate affects the current worth of future rewards. When $\gamma < 1$, the infinite sum converges to a finite value. With $\gamma = 0$, the agent maximizes immediate rewards. As γ approaches 1, future rewards carry more weight. We can also express the return $\mathcal{G}_t$ recursively

$$\mathcal{G}_t \equiv \mathcal{R}_{t+1} + \gamma \mathcal{G}_{t+1} \quad (15)$$

The return is finite if the reward is non-zero and constant, and $\gamma < 1$. Equation 16 works for both episodic and continuing tasks if $\mathcal{T} = \infty$ or $\gamma = 1$, respectively.

$$\mathcal{G}_t \equiv \sum_{k=t+1}^{\mathcal{T}} \gamma^{k-t-1} \mathcal{R}_k \quad (16)$$

The concepts introduced in this section—ranging from bandits and MDPs to value functions and policies—provide the mathematical and conceptual tools required to understand RL methods. In the next subsection, we explore how these foundational ideas evolve into core RL algorithms, bridging theory and application.

2.3 Policies and Value Functions

The value function estimates the expected return of the agent being in a certain state (or performing an action in a particular state). Depending on the actions selected, these factors will vary. There is a link between value functions and policies, which are linked to probabilities of action based on states. Value functions can be divided into two broad categories; **State Value Functions** and **Action Value Functions**. The value function of a state s under policy π , $v_{\pi}(s)$, is the expected return starting in s and following π

thereafter (Equation 17). On the other hand, the value of taking action a in state s under policy π , $q_\pi(s, a)$, is the expected return starting from s , taking action a , and following π thereafter (Equation 18).

$$v_\pi(s) \equiv \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k \mathcal{R}_{t+k+1} \mid \mathcal{S}_t = s \right], \quad \text{for all } s \in \mathcal{S} \quad (17)$$

$$q_\pi(s, a) \equiv \mathbb{E}_\pi [\mathcal{G}_t \mid \mathcal{S}_t = s, \mathcal{A}_t = a] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k \mathcal{R}_{t+k+1} \mid \mathcal{S}_t = s, \mathcal{A}_t = a \right] \quad (18)$$

It is important to note the difference between v and q , namely that q depends on the actions taken in each state. With ten states and eight actions per state, q requires 80 functions, while v requires only 10 functions. Following policy π , if an agent averages returns from each state, the average converges to $v_\pi(s)$. Averaging returns from each action converges to $q_\pi(s, a)$ [1]. $v_\pi(s)$ can be written recursively:

$$v_\pi(s) \equiv \mathbb{E}_\pi [\mathcal{G}_t \mid \mathcal{S}_t = s] = \mathbb{E}_\pi [\mathcal{R}_{t+1} + \gamma \mathcal{G}_{t+1} \mid \mathcal{S}_t = s] = \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r|s, a) [r + \gamma v_\pi(s')] \quad (19)$$

Equation 19 is the Bellman equation for v_π . The Bellman equation relates the value of a state to its potential successor states' values. The diagram illustrates the anticipation from a state to its successors. The value of the initial state equals the discounted value of the expected next state plus the anticipated reward [11, 1].

$v_\pi(s)$ and $q_\pi(s, a)$ serve different purposes in RL. In the evaluation of deterministic policies or when understanding the value of being in a particular state is required, **state-value functions** are used. In policy evaluation and policy iteration methods, where a policy is explicitly defined and it is necessary to evaluate the performance of being in a particular state under the policy, these methods are highly useful. The use of state-value functions is beneficial when there are many actions, since they reduce complexity by requiring only an evaluation of state values. **Action-value functions**, on the other hand, are used to evaluate and compare the potential for different actions when they are taking place in the same state. They are crucial for the selection of actions, where the goal is to determine the most appropriate action for each situation. As action-value functions take into account the expected return of different actions, they are particularly useful in environments with stochastic policies. Moreover, when dealing with continuous action spaces, action-value functions can provide a more detailed understanding of the impact of actions, aiding in the fine-tuning of policy implementation.

Example: Consider a gambling scenario where a player starts with \$10 and faces decisions regarding the amount to bet. This game illustrates state and action value functions in RL. **State Value function** ($v_\pi(s)$) quantifies expected cumulative future rewards for a state s , given policy π . Suppose the player has \$5:

- With a consistent \$1 bet, $v_\pi(5) = 0.5$ indicates an expected gain of \$0.5.
- With a consistent \$2 bet, $v_\pi(5) = -1$ indicates an expected loss of \$1.

Action Value function ($q_\pi(s, a)$) assesses expected cumulative future rewards for action a in state s . For instance:

- $q_\pi(5, 1) = 1$ suggests a \$1 bet from \$5 results in a cumulative reward of \$1.
- $q_\pi(5, 2) = -0.5$ indicates a loss of \$0.5 for a \$2 bet from \$5.

This gambling game scenario highlights the role of state and action value functions in RL, guiding optimal decision-making in dynamic environments.

2.4 Optimal Policies and Optimal Value Functions

Solving an RL task involves identifying a policy that maximizes long-term rewards. Value functions create a partial ordering over policies, allowing comparison and ranking based on expected cumulative rewards. A policy π is better than or equal to π_0 if $v_\pi(s) \geq v_{\pi_0}(s)$ for all states s . An optimal policy is better than or

equal to all other policies, denoted by π_* , sharing the same optimal state-value function v_* , which is defined as the maximum value function over all possible policies.

$$v_*(s) \equiv \max_{\pi} v_{\pi}(s) \quad \text{for all } s \in \mathcal{S} \quad (20)$$

Optimal policies also share the same optimal action-value function q_* , which is defined as the maximum action value function over all possible policies.

$$q_*(s, a) \equiv \max_{\pi} q_{\pi}(s, a) \quad \text{for all } s \in \mathcal{S} \quad (21)$$

The relationship between the optimal action value function $q_*(s, a)$ and the optimal state value function $v_*(s)$ is given by the following equation: By having optimal action value function $q_*(s, a)$ we can find optimal state value function $v_*(s)$ as shown in Equation 22.

$$q_*(s, a) = \mathbb{E}[\mathcal{R}_{t+1} + \gamma v_*(\mathcal{S}_{t+1}) \mid \mathcal{S}_t = s, \mathcal{A}_t = a] \quad (22)$$

Optimal value functions and policies represent an ideal state in RL. It is however rare to find truly optimal policies in computationally demanding tasks due to practical challenges [1]. RL agents strive to approximate optimal policies. Dynamic Programming (DP) helps identify optimal values, assuming a perfect model of the environment, which is a challenge to have in real-world cases. DP methods are not also sample efficient, even though they are theoretically sound. The fundamental idea of DP and RL is using value functions to organize the search for good policies. For finite MDPs, the environment's dynamics are given by probabilities $p(s', r \mid s, a)$. The Bellman optimality equations for the optimal state value function $v_*(s)$ and the optimal action value function $q_*(s, a)$ are Equations 23 and 24, respectively:

$$v_*(s) = \max_a \mathbb{E}[\mathcal{R}_{t+1} + \gamma v_*(\mathcal{S}_{t+1}) \mid \mathcal{S}_t = s, \mathcal{A}_t = a] = \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')] \quad (23)$$

$$q_*(s, a) = \mathbb{E}[\mathcal{R}_{t+1} + \max_{a'} q_*(\mathcal{S}_{t+1}, a') \mid \mathcal{S}_t = s, \mathcal{A}_t = a] = \sum_{s', r} p(s', r \mid s, a) [r + \gamma \max_{a'} q_*(s', a')] \quad (24)$$

DP algorithms are derived by transforming Bellman equations into update rules.

2.5 Policy Evaluation (Prediction)

Policy evaluation, also known as prediction, involves computing the state-value function v_{π} for a given policy π . This process assesses the expected return when following policy π from each state. The state-value function $v_{\pi}(s)$ is defined as the expected return starting from state s and following policy π :

$$v_{\pi}(s) \equiv \mathbb{E}_{\pi}[\mathcal{R}_{t+1} + \gamma \mathcal{G}_{t+1} \mid \mathcal{S}_t = s] \quad (25)$$

This can be recursively expressed as:

$$v_{\pi}(s) = \mathbb{E}_{\pi}[\mathcal{R}_{t+1} + \gamma v_{\pi}(\mathcal{S}_{t+1}) \mid \mathcal{S}_t = s] = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_{\pi}(s')] \quad (26)$$

In these equations, $\pi(a \mid s)$ denotes the probability of taking action a in state s under policy π . The existence and uniqueness of v_{π} are guaranteed if $\gamma < 1$ or if all states eventually terminate under π . Dynamic Programming (DP) algorithm updates are termed "*expected updates*" because they rely on the expectation over all potential next states, rather than just a sample [1].

2.6 Policy Improvement

The purpose of calculating the value function for a policy is to identify improved policies. Assuming v_π for a deterministic policy π , for a state s , should we alter the policy to select action $a \neq \pi(s)$? We know the effectiveness of adhering to the existing policy from state s ($v_\pi(s)$), but would transition to a new policy yield a superior outcome? We can answer this by selecting action a in s and then following π . To determine if a policy can be improved, we compare the value of taking a different action a in state s with the current policy. This is done using the action value function $q_\pi(s, a)$:

$$q_\pi(s, a) \equiv \mathbb{E}[\mathcal{R}_{t+1} + \gamma v_\pi(\mathcal{S}_{t+1}) \mid \mathcal{S}_t = s, \mathcal{A}_t = a] = \sum_{s', r} p(s', r \mid s, a)[r + \gamma v_\pi(s')] \quad (27)$$

The key criterion is whether this value exceeds $v_\pi(s)$. If $q_\pi(s, a) > v_\pi(s)$, consistently choosing action a in s is more advantageous than following π , leading to an improved policy π' .

2.7 Policy Improvement Theorem

The policy improvement theorem states that if $q_\pi(s, \pi'(s)) \geq v_\pi(s)$ for all states s , then the new policy π' is at least as good as the original policy π . Formally, it is expressed as

$$q_\pi(s, \pi'(s)) \geq v_\pi(s) \quad (28)$$

If π' achieves greater or equal expected return from all states $s \in \mathcal{S}$:

$$v_{\pi'}(s) \geq v_\pi(s) \quad (29)$$

If there is strict inequality at any state, π' is superior to π . Extending this to all states and actions, selecting the action that maximizes $q_\pi(s, a)$. The new policy π' is obtained by selecting the action that maximizes the action value function $q_\pi(s, a)$.

$$\pi'(s) \equiv \arg \max_a q_\pi(s, a) = \arg \max_a \mathbb{E}[\mathcal{R}_{t+1} + \gamma v_\pi(\mathcal{S}_{t+1}) \mid \mathcal{S}_t = s, \mathcal{A}_t = a] = \arg \max_a \sum_{s', r} p(s', r \mid s, a)[r + \gamma v_\pi(s')] \quad (30)$$

Policy improvement creates a new policy that enhances an initial policy by adopting a greedy approach based on the value function. Assuming π' is equally effective as π but not superior, then $v_\pi = v_{\pi'}$ ensures that for all states $s \in \mathcal{S}$. The relationship between the optimal state value function $v_*(s)$ and the optimal action value function $q_*(s, a)$ is given by the following equation:

$$v_{\pi'}(s) = \max_a \mathbb{E}[\mathcal{R}_{t+1} + \gamma v_{\pi'}(\mathcal{S}_{t+1}) \mid \mathcal{S}_t = s, \mathcal{A}_t = a] = \max_a \sum_{s', r} p(s', r \mid s, a)[r + \gamma v_{\pi'}(s')] \quad (31)$$

Policy improvement yields a superior policy unless the initial policy is already optimal. This concept extends to stochastic policies. Stochastic policies introduce a set of probabilities for actions, with the action most aligned with the greedy policy assigned the highest probability.

2.8 Policy Iteration

After enhancing a policy π using v_π to derive an improved policy π' , compute $v_{\pi'}$ and further refine it to obtain a superior policy π'' . This process generates a sequence of improving policies and corresponding value functions. The process of policy iteration involves alternating between policy evaluation and policy improvement to obtain a sequence of improving policies and value functions:

$$\pi_0 \xrightarrow{\text{Evaluation}} v_{\pi_0} \xrightarrow{\text{Improvement}} \pi_1 \xrightarrow{\text{Evaluation}} v_{\pi_1} \xrightarrow{\text{Improvement}} \pi_2 \xrightarrow{\text{Evaluation}} \dots \xrightarrow{\text{Improvement}} \pi_* \xrightarrow{\text{Evaluation}} v_* \quad (32)$$

Each policy in this sequence is a marked improvement over its predecessor unless the preceding one is already optimal. Given a finite MDP, this iterative process converges to an optimal policy and value function

in a finite number of iterations. This method is called Policy Iteration. Policy iteration involves two processes: policy evaluation aligns the value function with the current policy, and policy improvement makes the policy greedier based on the value function. These processes iteratively reinforce each other until an optimal policy is obtained.

2.9 Value Iteration

One limitation of policy iteration is that each iteration requires policy evaluation, often necessitating multiple passes through the entire state set [12]. To address this, policy evaluation can be abbreviated without losing convergence guarantees. This method, known as value iteration, terminates policy evaluation after a single sweep. It combines policy improvement with a truncated form of policy evaluation. Value iteration merges one pass of policy evaluation with policy improvement in each iteration, ensuring convergence to an optimal policy for discounted finite MDPs [13]. The update rule for value iteration is given in Equation 33.

$$v_{k+1}(s) \equiv \max_a \mathbb{E}[\mathcal{R}_{t+1} + \gamma v_k(\mathcal{S}_{t+1}) \mid \mathcal{S}_t = s, \mathcal{A}_t = a] = \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_k(s')] \quad (33)$$

In value iteration, the key advantage is its efficiency, as it reduces the computational burden by merging policy evaluation and improvement into a single update step. This method is particularly useful for large state spaces where full policy evaluation at each step of policy iteration is computationally prohibitive [1]. Additionally, value iteration can be implemented using a synchronous update approach, where all state values are updated simultaneously, or an asynchronous update approach, where state values are updated one at a time, potentially allowing for faster convergence in practice. Another notable aspect of value iteration is its robustness to initial conditions. Starting from an arbitrary value function, value iteration iteratively refines the value estimates until convergence, making it a reliable method for finding optimal policies even when the initial policy is far from optimal [14]. Furthermore, value iteration provides a foundation for more advanced algorithms by illustrating the principle of bootstrapping, where the value of a state is updated based on the estimated values of successor states. This principle is central to many RL algorithms that seek to balance exploration and exploitation in dynamic and uncertain environments [15].

The concepts introduced in this section—ranging from bandits and MDPs to value functions and policies—provide the mathematical and conceptual tools required to understand RL methods. In the next section, we explore how these foundational ideas evolve into core RL algorithms, bridging theory and application.

3 Core RL Methods

Understanding the various methodologies and concepts within RL is essential for the effective design and implementation of RL algorithms. Methods in RL can be classified as either off-policy or on-policy, and as model-free and model-based. These categories offer different approaches and techniques for learning from interactions with the environment.

3.1 Model-free & Model-based methods

Model-free methods determine the optimal policy or value function directly without constructing a model of the environment. There is no requirement for them to know transition probabilities and rewards, as they learn entirely from observed states, actions, and rewards. Compared with model-based methods, model-free methods are simpler to implement, relying on experience-based learning. There are two primary types: *Value-based* and *Policy-based* methods. The former focus on learning the action-value function to derive an optimal policy. For instance, Q-learning (discussed in section 4) is an off-policy algorithm that learns the value of the optimal policy independently of the agent’s actions by using a max operator in its update rule. SARSA (also discussed in section 4), on the other hand, is an on-policy algorithm that updates its Q-values based on the actions actually taken by the policy. Both methods update their action-value estimates based on the Bellman equation until convergence. In contrast, policy-based methods, like REINFORCE (discussed

in section 4), work by directly learning the policy without explicitly learning a value function. These methods adjust the policy parameters directly by following the gradient of the expected reward. This approach is particularly useful in environments with high-dimensional action spaces where value-based methods may not be effective. Policy-based methods are also capable of handling stochastic policies, providing a natural framework for dealing with uncertainty in action selection. In addition to these primary types, there are also **hybrid approaches** that combine value-based and policy-based methods, such as Actor-Critic algorithms (which will be discussed in section 4). These methods consist of two main components: an **actor** that updates the policy parameters in a direction suggested by the critic, and a **critic** that evaluates the action-value function. Combining both types of learning is intended to provide more stable and efficient learning [16].

Another significant advancement in model-free methods is the development of Deep RL (DRL). By integrating deep neural networks with traditional RL algorithms, methods such as Deep Q-Networks (DQN) [17] and Proximal Policy Optimization (PPO) [18] have achieved remarkable success in complex, high-dimensional environments, including games and robotic control tasks. The advancement of these technologies has opened up new possibilities for the application of RL to real-world problems, enabling the demonstration of robust performance in domains which were previously intractable. It is beyond the scope of this paper to discuss these algorithms, and we refer you to [19, 20, 21, 22] to understand DRL deeply and effectively.

It is possible to predict the outcomes of actions using **model-based** methods, which facilitate strategic planning and decision-making. The use of these methods enhances learning efficiency by providing opportunities for virtual experimentation, despite the complexity of developing and refining accurate models [7]. Autonomous driving systems are an example of how model-based methods can be applied in the real world. As autonomous vehicles navigate in dynamic environments, obstacle avoidance, and optimal routing must be made in real time. Autonomous vehicles create detailed models of their environment. These models include static elements, such as roads and buildings, as well as dynamic elements, such as other vehicles and pedestrians. Sensor data, including cameras, LIDAR, and radar, are used to build this model. Through the use of the environmental model, the vehicle is capable of predicting the outcome of various actions. For instance, when a vehicle considers changing lanes, it uses its model to predict the behavior of surrounding vehicles to determine the safest and most efficient way to make the change. The model assists the vehicle in planning its route and making strategic decisions. To minimize travel time, avoid congestion, and enhance safety, it evaluates different routes and actions. Simulation allows the vehicle to select the best course of action by simulating various scenarios before implementation in the real world. The vehicle, for example, may use the model to simulate different actions in the event of a busy intersection, such as waiting for a gap in traffic or taking an alternate route. Considering the potential outcomes of each action, the vehicle can make an informed decision that balances efficiency with safety. In addition to improving the ability of autonomous vehicles to navigate safely and efficiently in real-world conditions, this model-based approach enables them to make complex decisions with a high level of accuracy. As a result of continuously refining the model based on new data, the vehicle is able to enhance its decision-making capabilities over time, thereby improving performance and enhancing safety on the road.

There are several advantages to using model-based methods over methods that do not use models. By simulating future states and rewards, they can plan and evaluate different action sequences without interacting directly with the environment. It is believed that this capability may lead to a faster convergence to an optimal policy, since learning can be accelerated by leveraging the model's predictions. A model-based approach can also adapt more quickly to changes in the environment, since it enables the model to be updated and re-planned accordingly. Although model-based methods have many advantages, they also face a number of challenges, primarily in regards to accuracy and computational cost. In order to create an accurate model of the environment, a high-fidelity model needs to be created. Moreover, the planning process may be computationally expensive, especially in environments with a large number of states and actions. However, advances in computing power and algorithms continue to improve the feasibility and performance of model-based methods, making them a valuable approach in RL [23].

3.2 Off-Policy and On-Policy Methods

On-policy and off-policy learning are methodologies within model-free learning approaches, not relying on environment transition probabilities. They are classified based on the relationship between the behavior policy and the updated policy [1]. **On-policy** methods evaluate and improve the policy used to make decisions, intertwining exploration and learning. These methods update the policy based on the actions taken and the rewards received while following the current policy (π). This ensures that the policy being optimized is the one actually used to interact with the environment, allowing for a coherent learning process where exploration and policy improvement are naturally integrated.

Off-policy methods, on the other hand, involve learning the value of the optimal policy independently of the agent's actions. In these methods, we distinguish between two types of policies: the *behavior policy* (b) and the *target policy* (π). The behavior policy explores the environment, while the target policy aims to improve performance based on the gathered experience. This allows for a more exploratory behavior policy while learning an optimal target policy. A significant advantage of off-policy methods is that they can learn from data generated by any policy, not just the one currently being followed, making them highly flexible and sample-efficient. The decoupling of the behavior and target policies allows off-policy methods to reuse experiences more effectively. For instance, experiences collected using a behavior policy that explores the environment broadly can be used to improve the target policy, which aims to maximize rewards. This characteristic makes off-policy methods particularly powerful in dynamic and complex environments where extensive exploration is required [24, 25].

The relationship between the target policy and the behavior policy determines if a method is on-policy or off-policy. Identical policies indicate on-policy, while differing policies indicate off-policy. Implementation details and objectives also influence classification. To better distinguish these methods, we have to first learn what are the different policies. **Behavior Policy** b is a strategy used by an agent to determine which actions to take at each time step. The behavior policy might, for example, include recommending a variety of movies in order to explore user preferences in the recommendation system example. **Target policy** π governs how the agent updates its value estimates in response to observed outcomes. Depending on the feedback received from the recommended movies, the target policy of the recommendation system may update the estimated user preferences. A thorough understanding of the interactions between these policies is essential for the implementation of effective learning systems. An agent's behavior policy determines how it explores an environment, balancing exploration with exploitation to gather useful information. Alternatively, the target policy determines how the agent learns from these experiences in order to improve its estimates of value. When using on-policy methods, the behavior policy and the target policy are the same, meaning that the actions taken to interact with the environment are also used to update the value estimates. The result is stable learning, but it can be less efficient because the policy may not sufficiently explore the state space [26]. There is a difference between the behavior policy and the target policy in off-policy methods. As opposed to the behavior policy, the target policy focuses on optimizing the value estimates by taking the most appropriate action. Despite the fact that this separation can make learning more efficient, it can also introduce instability if the behavior policy diverges too far from the optimal policy [15, 25]. Furthermore, advanced methods, such as Actor-Critic algorithms, separate the *behavior policy* (*actor*) and the *target policy* (*critic*). Actors make decisions according to current policies, while critics evaluate these decisions and provide feedback to improve policies, thus combining the stability of on-policy methods with the efficiency of off-policy methods [27, 28].

Understanding the core methodologies of RL, such as model-free and model-based approaches, as well as the distinction between off-policy and on-policy methods, provides a foundational framework for exploring the diverse landscape of RL algorithms. These methodologies not only shape how agents learn from their environments but also influence their adaptability and efficiency in complex, real-world scenarios. Building on this understanding, the next section delves deeper into the specific essential algorithms underpinning these methods, focusing on the policy-based, value-based, and hybrid approaches, to provide a clearer picture of their mechanisms and applications in RL.

4 Essential Algorithms

This section aims to present a concise overview of key algorithms discussed thus far, accompanied by references to the original research papers for further exploration. Each algorithm is briefly described, and real-world examples are included to enhance understanding. Readers seeking detailed information are encouraged to consult the cited references, which serve as a gateway to the primary sources and support a deeper learning experience. We categorize algorithms into three types: Value-based, Policy-based, and Hybrid Algorithms. For each type, we analyze one to two widely-used algorithms, acknowledging that there are more algorithms to discover.

4.1 Value-based

We introduced value-based methods, and clearly analyzed how they work. Here, we introduce three algorithms, that are in the tabular settings. Later, we dive deeper into the topics, and discuss value-based methods that use Deep Learning, such as Deep Q-Networks.

A significant breakthrough was made by [29] with the introduction of **Q-learning**, a Model-free algorithm considered as off-policy Temporal Difference (TD) control. TD learning is undoubtedly the most fundamental and innovative concept. A combination of Monte Carlo (MC) methods and Dynamic Programming (DP) is used in this method. On one hand, similar to MC approaches, TD learning can be used to acquire knowledge from unprocessed experience without the need for a model that describes the dynamics of the environment. On the other hand, TD algorithms are also similar to DP in that they refine predictions using previously learned estimates instead of requiring a definitive outcome in order to proceed (known as *bootstrapping*). Q-learning enables an agent to learn the value of an action in a particular state through experience, without requiring a model of the environment. It operates on the principle of learning an action-value function that gives the expected utility of taking a given action in each state and following a fixed policy thereafter. The core of the Q-learning algorithm involves updating the Q-values (action-value pairs), where the learned action-value function, denoted as Q , approximates q_* , the optimal action-value function, regardless of the policy being followed. This significantly simplifies the algorithm's analysis and has facilitated early proofs of convergence. However, the policy still influences the process by determining which state-action pairs are visited and subsequently updated.

$$Q(\mathcal{S}_t, \mathcal{A}_t) \leftarrow Q(\mathcal{S}_t, \mathcal{A}_t) + \alpha [\mathcal{R}_{t+1} + \gamma \max_a Q(\mathcal{S}_{t+1}, a) - Q(\mathcal{S}_t, \mathcal{A}_t)] \quad (34)$$

There are other types of Q-learning introduced in the literature, with slight changes and improvements, such as: Double Q-learning [30], that addresses the overestimation bias in Q-learning, Distributional Q-learning [31], which models the distribution of returns instead of estimating the mean Q-value, providing richer information for decision-making, and many more [32, 33, 34].

Another widely used value-based algorithm is **Deep Q-Networks (DQN)**, which merges *Q-learning* with Neural Networks to learn control policies directly from raw pixel inputs. It uses Convolutional Neural Networks (CNN) to process these inputs and an experience replay mechanism to stabilize learning by breaking correlations between consecutive experiences. The target network, updated less frequently, aids in stabilizing training. DQN achieved state-of-the-art performance on various Atari 2600 games, surpassing previous methods and, in some cases, human experts, using a consistent network architecture and hyperparameters across different games [17]. DQN combines the introduced Bellman Equation with DL approaches like Loss Function and Gradient Descent to find the optimal policy as below:

$$\mathcal{L}_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[(y_i - Q(s, a; \theta_i))^2 \right] \quad (35)$$

where

$$y_i = r + \gamma \max_{a'} Q(s', a'; \theta^-) \quad (36)$$

$$\nabla_{\theta_i} \mathcal{L}_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta_i) \right) \nabla_{\theta_i} Q(s, a; \theta_i) \right] \quad (37)$$

Similar to Q-learning, there have been updates made to DQN. Some of the variations are [35, 36, 37].

Table 1: Essential RL Algorithms

Algorithms	
1	Q-Learning [29] - Model-free, Off-policy, Value-based
2	SARSA (State-Action-Reward-State-Action) [26] - Model-free, On-policy, Value-based
3	Expected SARSA [38] - Model-free, On-policy, Value-based
4	REINFORCE [39] - Model-free, On-policy, Policy-based
5	Dyna-Q [40] - Model-based, Off-policy, Hybrid
6	DQN [17] - Model-free, Off-policy, Value-based
7	TRPO [41] - Model-free, On-policy, Policy-based
8	PPO [18] - Model-free, On-policy, Policy-based
9	SAC (Soft Actor-Critic) [33] - Model-free, Off-policy, Hybrid
10	A3C [42] - Model-free, On-policy, Hybrid
11	A2C [28] - Model-free, On-policy, Hybrid
12	DDPG (Deep Deterministic Policy Gradient) [28] - Model-free, Off-policy, Policy-based
13	TD3 (Twin Delayed Deep Deterministic Policy Gradient) [43] - Model-free, Off-policy, Policy-based

4.2 Policy-based

Moving on from value-based methods, we analyze some policy-based algorithms in this section. **Policy-based** methods are another fundamental RL method that more strongly emphasizes direct policy optimization in the process of choosing actions for an agent. In contrast to Value-based methods, which search for the value function implicit in the task, and then derive an optimal policy, Policy-based methods directly parameterize and optimize the policy. This approach offers several advantages, particularly better dealing with very challenging environments that have high-dimensional action spaces or where policies are inherently stochastic. Perhaps at the core, Policy-based methods conduct their operation based on the parameterization of policies, usually denoted as $\pi(a|s; \theta)$. Here, θ is used to denote the parameters of the policy, while s denotes the state and a denotes the action. In other words, it finds the optimal parameters θ^* that maximize the expected cumulative reward. Needless to say, this is generally done by gradient ascent techniques and more specifically by Policy Gradient methods that explicitly compute the gradient of expected reward with respect to the policy parameters, modifying parameters in the direction of reward increase [1, 10, 44].

REINFORCE is one of the widely-used policy-based algorithms. The REINFORCE algorithm is a seminal contribution to RL, particularly within the context of policy gradient methods. The algorithm is designed to optimize the expected cumulative reward by adjusting the policy parameters in the direction of the gradient of the expected reward. It is rooted in the stochastic policy framework, where the policy, parameterized by θ , defines a probability distribution over actions given the current state. The key insight of the REINFORCE algorithm is to use the log-likelihood gradient estimator to update the policy parameters [39]. The gradient of the expected reward with respect to the policy parameters θ is given by:

$$\nabla_{\theta} \mathcal{J}(\theta) = \mathbb{E}_{\pi} [\nabla_{\theta} \log \pi_{\theta}(a|s) \mathcal{G}_t], \quad (38)$$

where $\pi_{\theta}(a|s)$ is the probability of taking action a in the state s under policy π parameterized by θ , and $\mathcal{G}_t$ is the return (cumulative future reward) following time step t . This gradient estimation forms the basis for the parameter update rule.

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(a|s) \mathcal{G}_t, \quad (39)$$

where α is the learning rate.

Another algorithm, which has been used in variety of applications [45, 46, 47, 48, 49, 50, 51, 52], is **Proximal Policy Optimization (PPO)**. PPO, proposed by [18], represents a significant advancement within policy gradient methods. PPO aims to achieve reliable performance and sample efficiency, addressing the limitations of previous policy optimization algorithms such as Vanilla Policy Gradient (VPG) [53] methods and Trust Region Policy Optimization (TRPO) [41]. Using policy gradient methods, the policy parameters

are optimized through stochastic gradient ascent by estimating the gradient of the policy. One of the most commonly used policy gradient estimators is:

$$\hat{g} = \hat{\mathbb{E}}_t \left[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t \right], \quad (40)$$

where π_{θ} represents the policy parameterized by θ , and $\hat{A}_t$ is an estimator of the advantage function at time step t . This estimator helps construct an objective function whose gradient corresponds to the policy gradient estimator:

$$\mathcal{L}_{\mathcal{PG}}(\theta) = \hat{\mathbb{E}}_t \left[\log \pi_{\theta}(a_t | s_t) \hat{A}_t \right]. \quad (41)$$

PPO simplifies TRPO by using a surrogate objective with a clipped probability ratio, allowing for multiple epochs of mini-batch updates. In order to preserve learning, large policy updates should be avoided.

4.3 Hybrid (Actor-Critic) methods

For the last group of algorithm, hybrid methods, we introduce **Asynchronous Advantage Actor-Critic (A3C)** and **Advantage Actor-Critic (A2C)**. Actor-critic methods combine Value-based and Policy-based approaches. Essentially, these methods consist of two components: the Actor, who selects actions based on a policy, and the Critic, who evaluates the actions based on their value function. By providing feedback on the quality of the actions taken, the critic guides the actor in updating the policy directly. As a result of this synergy, learning can be more stable and efficient, addressing some limitations of pure policy or Value-based approaches [16, 54]. The A2C algorithm is a synchronous variant of the A3C algorithm, which was introduced by [42]. A2C maintains the key principles of A3C but simplifies the training process by synchronizing the updates of multiple agents, thereby leveraging the strengths of both Actor-Critic methods and advantage estimation. The Actor-Critic architecture combines two primary components, in both algorithms: the actor, which is responsible for selecting actions, and the critic, which evaluates the actions by estimating the value function. The actor updates the policy parameters in a direction that is expected to increase the expected reward, while the critic provides feedback by computing the TD error. This integration allows for more stable and efficient learning compared to using Actor-only or critic-only methods [27]. Advantage estimation is a technique used to reduce the variance of the policy gradient updates. The advantage function $\mathcal{A}(s, a)$ represents the difference between the action-value function $\mathcal{Q}(s, a)$ and the value function $\mathcal{V}(s)$.

$$\mathcal{A}(s, a) = \mathcal{Q}(s, a) - \mathcal{V}(s). \quad (42)$$

By using the advantage function, A2C focuses on actions that yield higher returns than the average, which helps in making more informed updates to the policy [1]. Unlike A3C, where multiple agents update the global model asynchronously, A2C synchronizes these updates. Multiple agents run in parallel environments, collecting experiences and calculating gradients, which are then aggregated and used to update the global model synchronously. This synchronization reduces the complexity of implementation and avoids issues related to asynchronous updates, such as non-deterministic behavior and potential overwriting of gradients. Table 1 categorizes the examined algorithms, and other essential algorithms that we did not discuss to give a comprehensive overview regarding the main features

By analyzing some of the widely-used algorithms, it is time to introduce some of the good resources to further learn RL. In the next section, we introduce books, video lectures, and online communities in RL.

5 Resources and Further Reading

In this section, we provide a list of some of the most helpful books, courses, videos, and online communities to assist readers in getting started with their real-life research without being overwhelmed. Table 2 summarizes the necessary resources, all in one place. On top of the mentioned resources, we would like to refer readers to survey papers [10, 19, 54, 55, 56, 23, 57, 7, 58, 59] that are really helpful to understand different applications, algorithms, and background more.

Table 2: Reinforcement Learning Resources

Books	
1	"Reinforcement Learning: An Introduction" by Richard S. Sutton and Andrew G. Barto
2	"Deep Reinforcement Learning Hands-On" by Maxim Lapan
3	"Grokking Deep Reinforcement Learning" by Miguel Morales
4	"Algorithms for Reinforcement Learning" by Csaba Szepesvári
Online Courses	
5	Coursera: Reinforcement Learning Specialization by University of Alberta
6	Udacity: Deep Reinforcement Learning Nanodegree
7	edX: Fundamentals of Reinforcement Learning by University of Alberta
8	Reinforcement Learning Winter 2019 (Stanford)
Video Lectures	
9	DeepMind x UCL — Reinforcement Learning Lecture Series
10	David Silver's Reinforcement Learning Course
11	Pascal Poupart's Reinforcement Learning Course - CS885
12	Sarath Chandar's Reinforcement Learning Course
Tutorials and Articles	
13	OpenAI Spinning Up in Deep RL
14	Deep Reinforcement Learning Course by PyTorch
15	RL Adventure by Denny Britz
Online Communities and Forums	
16	Reddit: r/reinforcementlearning
17	Stack Overflow
18	AI Alignment Forum

6 Conclusion

This paper presents an introductory exploration of the fundamental concepts and methodologies of Reinforcement Learning (RL), tailored to beginners. It establishes a foundational understanding of how RL agents learn and make decisions by thoroughly examining key components such as states, actions, policies, and reward signals. By analyzing Multi-armed bandit problem, this paper introduced background of RL in an accessible and easy-to-understand way. The primary objective is to offer an overview of a wide range of RL algorithms, encompassing both model-free and model-based approaches, thereby highlighting the diversity within the field. Through this guide, we aim to equip new learners with the essential knowledge and confidence to begin their journey into RL.

References

- [1] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
- [2] M. Li, C. Shi, Z. Wu, and P. Fryzlewicz, "Testing stationarity and change point detection in reinforcement learning," *arXiv preprint arXiv:2203.01707*, 2022.
- [3] S. B. Thrun, *Efficient exploration in reinforcement learning*. Carnegie Mellon University, 1992.
- [4] P. Erd, "On a new law of large numbers," *J. Anal. Muth*, vol. 22, pp. 103–1, 1970.
- [5] A. Garivier and E. Moulines, "On upper-confidence bound policies for switching bandit problems," in *International conference on algorithmic learning theory*, pp. 174–188, Springer, 2011.
- [6] P. Ladosz, L. Weng, M. Kim, and H. Oh, "Exploration in deep reinforcement learning: A survey," *Information Fusion*, vol. 85, pp. 1–22, 2022.

- [7] F.-M. Luo, T. Xu, H. Lai, X.-H. Chen, W. Zhang, and Y. Yu, "A survey on model-based reinforcement learning," *Science China Information Sciences*, vol. 67, no. 2, p. 121101, 2024.
- [8] M. A. Wiering and M. Van Otterlo, "Reinforcement learning," *Adaptation, learning, and optimization*, vol. 12, no. 3, p. 729, 2012.
- [9] M. Van Otterlo and M. Wiering, "Reinforcement learning and markov decision processes," in *Reinforcement learning: State-of-the-art*, pp. 3–42, Springer, 2012.
- [10] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *Journal of artificial intelligence research*, vol. 4, pp. 237–285, 1996.
- [11] B. O'Donoghue, I. Osband, R. Munos, and V. Mnih, "The uncertainty bellman equation and exploration," in *International conference on machine learning*, pp. 3836–3845, 2018.
- [12] D. P. Bertsekas, "Approximate policy iteration: A survey and some new methods," *Journal of Control Theory and Applications*, vol. 9, no. 3, pp. 310–335, 2011.
- [13] M. Lutter, S. Mannor, J. Peters, D. Fox, and A. Garg, "Value iteration in continuous actions, states and time," *arXiv preprint arXiv:2105.04682*, 2021.
- [14] D. Bertsekas, *Dynamic programming and optimal control: Volume I*, vol. 4. Athena scientific, 2012.
- [15] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, *et al.*, "Human-level control through deep reinforcement learning," *nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [16] I. Grondman, L. Busoniu, G. A. Lopes, and R. Babuska, "A survey of actor-critic reinforcement learning: Standard and natural policy gradients," *IEEE Transactions on Systems, Man, and Cybernetics, part C (applications and reviews)*, vol. 42, no. 6, pp. 1291–1307, 2012.
- [17] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," *arXiv preprint arXiv:1312.5602*, 2013.
- [18] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [19] Y. Li, "Deep reinforcement learning: An overview," *arXiv preprint arXiv:1701.07274*, 2017.
- [20] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "A brief survey of deep reinforcement learning," *arXiv preprint arXiv:1708.05866*, 2017.
- [21] V. François-Lavet, P. Henderson, R. Islam, M. G. Bellemare, J. Pineau, *et al.*, "An introduction to deep reinforcement learning," *Foundations and Trends® in Machine Learning*, vol. 11, no. 3-4, pp. 219–354, 2018.
- [22] S. E. Li, "Deep reinforcement learning," in *Reinforcement learning for sequential decision and optimal control*, pp. 365–402, Springer, 2023.
- [23] T. M. Moerland, J. Broekens, A. Plaat, C. M. Jonker, *et al.*, "Model-based reinforcement learning: A survey," *Foundations and Trends® in Machine Learning*, vol. 16, no. 1, pp. 1–118, 2023.
- [24] P. Dayan and C. Watkins, "Q-learning," *Machine learning*, vol. 8, no. 3, pp. 279–292, 1992.
- [25] M. Hessel, J. Modayil, H. Van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, and D. Silver, "Rainbow: Combining improvements in deep reinforcement learning," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, 2018.
- [26] G. A. Rummery and M. Niranjan, *On-line Q-learning using connectionist systems*, vol. 37. University of Cambridge, Department of Engineering Cambridge, UK, 1994.

- [27] V. Konda and J. Tsitsiklis, "Actor-critic algorithms," *Advances in neural information processing systems*, vol. 12, 1999.
- [28] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.
- [29] C. J. C. H. Watkins, "Learning from delayed rewards," 1989.
- [30] H. Hasselt, "Double q-learning," *Advances in neural information processing systems*, vol. 23, 2010.
- [31] M. G. Bellemare, W. Dabney, and R. Munos, "A distributional perspective on reinforcement learning," in *International conference on machine learning*, pp. 449–458, PMLR, 2017.
- [32] L. Matignon, G. J. Laurent, and N. Le Fort-Piat, "Hysteretic q-learning: an algorithm for decentralized reinforcement learning in cooperative multi-agent teams," in *2007 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 64–69, IEEE, 2007.
- [33] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International conference on machine learning*, pp. 1861–1870, PMLR, 2018.
- [34] J. Wang, Z. Ren, T. Liu, Y. Yu, and C. Zhang, "Qplex: Duplex dueling multi-agent q-learning," *arXiv preprint arXiv:2008.01062*, 2020.
- [35] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 30, 2016.
- [36] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling network architectures for deep reinforcement learning," in *International conference on machine learning*, pp. 1995–2003, PMLR, 2016.
- [37] T. Schaul, "Prioritized experience replay," *arXiv preprint arXiv:1511.05952*, 2015.
- [38] H. Van Seijen, H. Van Hasselt, S. Whiteson, and M. Wiering, "A theoretical and empirical analysis of expected sarsa," in *2009 IEEE symposium on adaptive dynamic programming and reinforcement learning*, pp. 177–184, IEEE, 2009.
- [39] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine learning*, vol. 8, pp. 229–256, 1992.
- [40] R. S. Sutton, "Integrated architectures for learning, planning, and reacting based on approximating dynamic programming," in *Machine learning proceedings 1990*, pp. 216–224, Elsevier, 1990.
- [41] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *International conference on machine learning*, pp. 1889–1897, PMLR, 2015.
- [42] V. Mnih, "Asynchronous methods for deep reinforcement learning," *arXiv preprint arXiv:1602.01783*, 2016.
- [43] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International conference on machine learning*, pp. 1587–1596, PMLR, 2018.
- [44] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *The International Journal of Robotics Research*, vol. 32, no. 11, pp. 1238–1274, 2013.
- [45] H. Wei, X. Liu, L. Mashayekhy, and K. Decker, "Mixed-autonomy traffic control with proximal policy optimization," in *2019 IEEE Vehicular Networking Conference (VNC)*, pp. 1–8, IEEE, 2019.
- [46] B. Zhang, X. Lu, R. Diao, H. Li, T. Lan, D. Shi, and Z. Wang, "Real-time autonomous line flow control using proximal policy optimization," in *2020 IEEE Power & Energy Society General Meeting (PESGM)*, pp. 1–5, IEEE, 2020.

- [47] J. Jin and Y. Xu, "Optimal policy characterization enhanced proximal policy optimization for multitask scheduling in cloud computing," *IEEE Internet of Things Journal*, vol. 9, no. 9, pp. 6418–6433, 2021.
- [48] L. Zhang, Y. Zhang, X. Zhao, and Z. Zou, "Image captioning via proximal policy optimization," *Image and Vision Computing*, vol. 108, p. 104126, 2021.
- [49] Y. Guan, Y. Ren, S. E. Li, Q. Sun, L. Luo, and K. Li, "Centralized cooperation for connected and automated vehicles at intersections by proximal policy optimization," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 11, pp. 12597–12608, 2020.
- [50] E. Böhn, E. M. Coates, S. Moe, and T. A. Johansen, "Deep reinforcement learning attitude control of fixed-wing uavs using proximal policy optimization," in *2019 international conference on unmanned aircraft systems (ICUAS)*, pp. 523–533, IEEE, 2019.
- [51] G. C. Lopes, M. Ferreira, A. da Silva Simões, and E. L. Colombini, "Intelligent control of a quadrotor with proximal policy optimization reinforcement learning," in *2018 Latin American Robotic Symposium, 2018 Brazilian Symposium on Robotics (SBR) and 2018 Workshop on Robotics in Education (WRE)*, pp. 503–508, IEEE, 2018.
- [52] F. Ye, X. Cheng, P. Wang, C.-Y. Chan, and J. Zhang, "Automated lane change strategy using proximal policy optimization-based deep reinforcement learning," in *2020 IEEE Intelligent Vehicles Symposium (IV)*, pp. 1746–1752, IEEE, 2020.
- [53] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," *Advances in neural information processing systems*, vol. 12, 1999.
- [54] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 26–38, 2017.
- [55] X. Wang, S. Wang, X. Liang, D. Zhao, J. Huang, X. Xu, B. Dai, and Q. Miao, "Deep reinforcement learning: A survey," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 4, pp. 5064–5078, 2022.
- [56] H.-n. Wang, N. Liu, Y.-y. Zhang, D.-w. Feng, F. Huang, D.-s. Li, and Y.-m. Zhang, "Deep reinforcement learning: a survey," *Frontiers of Information Technology & Electronic Engineering*, vol. 21, no. 12, pp. 1726–1744, 2020.
- [57] A. S. Polydoros and L. Nalpantidis, "Survey of model-based reinforcement learning: Applications on robotics," *Journal of Intelligent & Robotic Systems*, vol. 86, no. 2, pp. 153–173, 2017.
- [58] Y. Sato, "Model-free reinforcement learning for financial portfolios: a brief survey," *arXiv preprint arXiv:1904.04973*, 2019.
- [59] J. Ramírez, W. Yu, and A. Perrusquía, "Model-free reinforcement learning from expert demonstrations: a survey," *Artificial Intelligence Review*, vol. 55, no. 4, pp. 3213–3241, 2022.